



Università
di Catania

DISTRIBUTED SYSTEMS AND BIG DATA
A.A. 2025/26

Homework 3 DSBD

STUDENTI:

Alessia Fichera 1000084787

Davide Pantò 1000081854

Abstract

Rispetto alla versione precedente, l'architettura del sistema è stata evoluta introducendo la containerizzazione completa di tutti i microservizi tramite Docker e l'adozione di Kubernetes per l'orchestrazione, implementato localmente tramite Kind. Parallelamente, è stato integrato un meccanismo di White-box Monitoring basato su Prometheus: invece di limitarsi a verificare la raggiungibilità dei servizi, il sistema raccoglie ora metriche applicative interne (come il numero di voli processati o i tempi di esecuzione) esposte direttamente dai microservizi tramite endpoint HTTP dedicati, garantendo osservabilità in tempo reale.

Kubernetes

Kubernetes è il sistema open-source utilizzato per l'orchestrazione dei container. Esso automatizza il deployment, lo scaling e la gestione delle applicazioni containerizzate, gestendo il bilanciamento del carico e garantendo che lo stato desiderato del cluster corrisponda a quello attuale.

Kind (Kubernetes IN Docker)

Per l'ambiente di sviluppo locale è stato utilizzato **Kind**, come evidenziato dalla configurazione del cluster. Kind permette di eseguire un cluster Kubernetes locale utilizzando container Docker come nodi del cluster, riducendo il consumo di risorse e semplificando la configurazione delle porte (es. mappatura porte 30080 e 30081 per l'accesso ai servizi).

Oggetti Kubernetes Implementati

Per orchestrare l'applicazione sono stati definiti i seguenti oggetti Kubernetes nei file YAML

Deployments e Services Sono stati creati Deployments per garantire la replicazione e il ciclo di vita dei microservizi, e Services per esporre gli endpoint di rete stabili:

- **Microservizi Applicativi:**
 - usermanager: Gestisce la logica utente e l'autenticazione.
 - datacollector: Responsabile del download dei dati e dell'esposizione delle metriche.
 - alert-system e notifier-system: Gestiscono la logica di alerting e l'invio delle notifiche.

- **Infrastruttura:**

- mysql, redis, kafka: Servizi di supporto per la persistenza e la messaggistica.
- nginx: API Gateway esposto tramite NodePort (30080) per l'instradamento delle richieste.

ConfigMaps e Secrets

- **Secrets (app-secrets):** Utilizzati per iniettare in modo sicuro le credenziali sensibili, come le password del database e le chiavi API di OpenSky, all'interno dei container.
- **ConfigMaps (nginx-config, prometheus-config):** Utilizzate per disaccoppiare i file di configurazione dal codice, ad esempio per definire le regole di routing di Nginx o i target di scraping di Prometheus.

Prometheus e Monitoraggio (White-box)

Il sistema integra **Prometheus** per il monitoraggio attivo. A differenza del black-box monitoring (che verifica solo se un servizio risponde), il **white-box monitoring** espone lo stato interno dell'applicazione tramite metriche personalizzate.

1. **Configurazione Scraping:** Prometheus è configurato per eseguire lo scraping periodico (ogni 15s) dei target. Nello specifico, Prometheus visita periodicamente l'endpoint **9999** esposto dai microservizi:
 - **DataCollector:** per raccogliere i dati operativi sui voli.
 - **UserManager:** per monitorare le interazioni degli utenti.

Il sistema "legge" i numeri aggiornati in quel preciso istante e li archivia per permetterti poi di creare grafici o generare allarmi.

2. **Metriche Implementate (Data Collector):** Per garantire un'osservabilità granulare sul processo di acquisizione dati da OpenSky, il microservizio implementa due metriche custom fondamentali:
 - **datacollector_download_data (Tipologia: Gauge)** Questa metrica misura i valori istantanei relativi all'ultimo ciclo di download. Viene utilizzata con diverse etichette (resource) per monitorare:
 - **Latenza (total_time):** Il tempo esatto, in secondi, impiegato per completare la richiesta API.
 - **Ampiezza del Batch (last_n_voli):** Il carico di lavoro istantaneo, ovvero quanti voli sono stati recuperati nell'ultima iterazione.
 - **datacollector_num_flights_downloaded (Tipologia: Counter)** Un contatore incrementale che traccia il numero totale di voli scaricati dall'avvio del servizio (total_n_voli). Questa metrica è fondamentale per

monitorare il volume di dati processati nel tempo e rilevare eventuali interruzioni nel flusso di dati.

3. Metriche Implementate (User Manager) Parallelamente, il microservizio User Manager espone metriche dedicate per analizzare le prestazioni delle operazioni di gestione account (Login, Registrazione, Cancellazione):

- **usermanager_login_response_time_seconds (Tipologia: Gauge):** Misurano il tempo di esecuzione dell'ultima richiesta, permettendo di identificare rallentamenti nel database o nella logica di hashing.
- **usermanager_login_attempts_total (Tipologia: Counter):** Tengono traccia del volume totale delle richieste, distinguendo tramite etichette (*labels*) tra operazioni riuscite ed errori.

Diagramma Architettuale:

Il diagramma rappresenta l'architettura dell'applicazione, mostrando le principali interazioni tra i microservizi:

- **Data Collector:** Micro Servizio che raccoglie in modo continuativo le informazioni sui voli dall'API esterna OpenSky Network, aggiornando nelle tabelle del DataDB.
- **User Manager:** Micro servizio che gestisce le richieste di registrazione, login e cancellazione dell'utente nello UserDB.
- **gRPC Service User:** Verifica l'autenticazione prima di processare richieste sui voli.
- **gRPC Service Data:** Rimuove gli interessi dell'utente quando viene cancellato dal sistema.
- **DataDB:** Contiene le tabelle con le informazioni sui voli.
- **UserDB:** Contiene le tabelle con le informazioni sugli utenti.
- **Redis:** funge da cache temporanea per intercettare i tentativi duplicati di registrazione ed evitare interrogazioni inutili al database.
- **API Gateway (Nginx):** Punto di ingresso unico che instrada le richieste REST verso i microservizi competenti (User Manager o Data Collector).
- **Broker Kafka:** Bus di messaggistica asincrono che disaccoppia la raccolta dati dalla pipeline di alerting, distribuendo i trigger di aggiornamento.
- **Alert System:** Componente Consumer/Producer che verifica le soglie nel DB e genera gli eventi di allarme.
- **Alert Notifier System:** Consumer dedicato esclusivamente all'invio delle notifiche email agli utenti.

Componente aggiuntiva:

- **Prometheus:** Sistema di monitoraggio white-box che esegue lo scraping periodico delle metriche esposte dai microservizi. Interroga la porta 9999 sia del Data Collector (metriche voli) che dello User Manager (metriche utenti), centralizzando la raccolta dati per analisi in tempo reale.

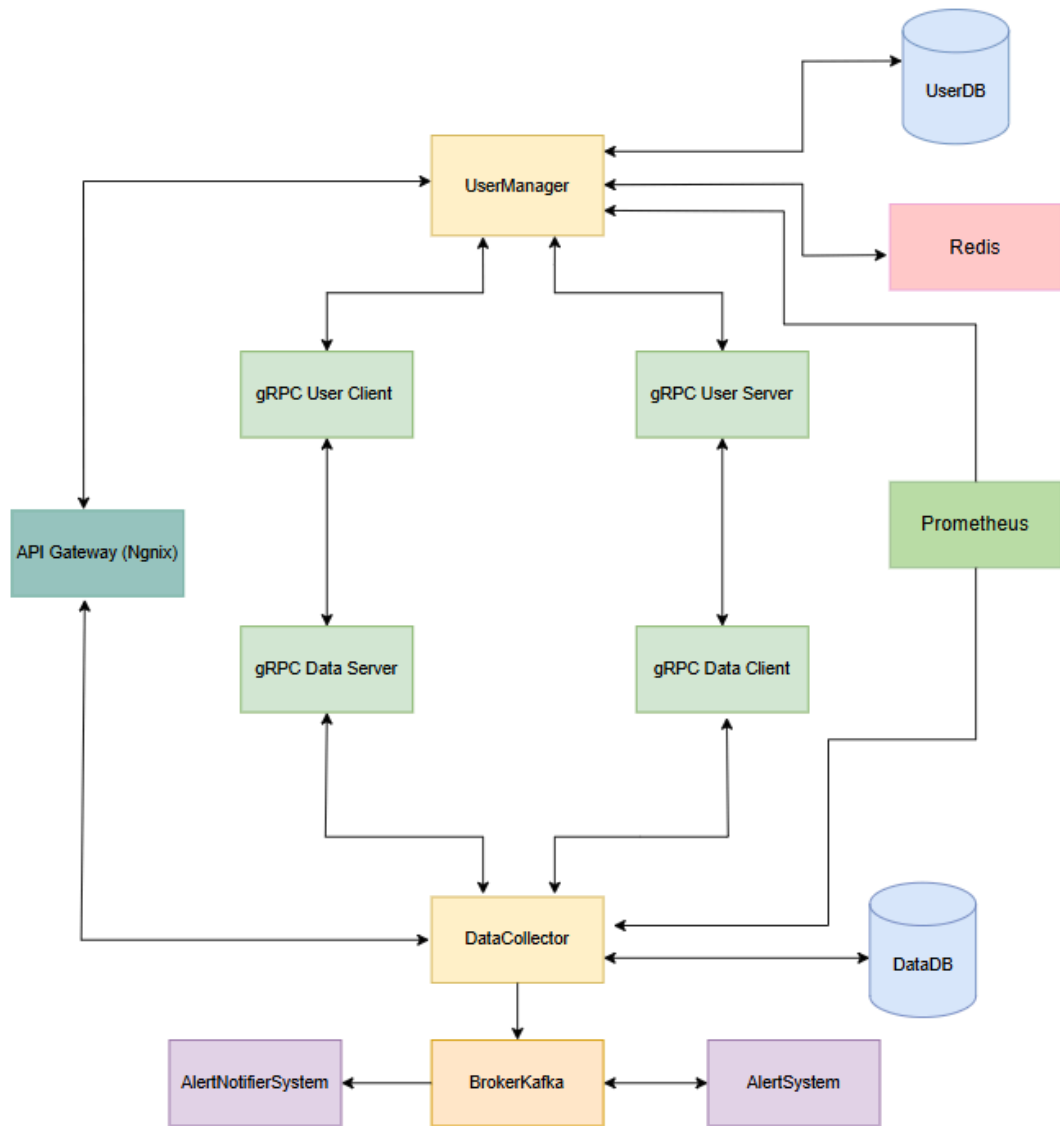


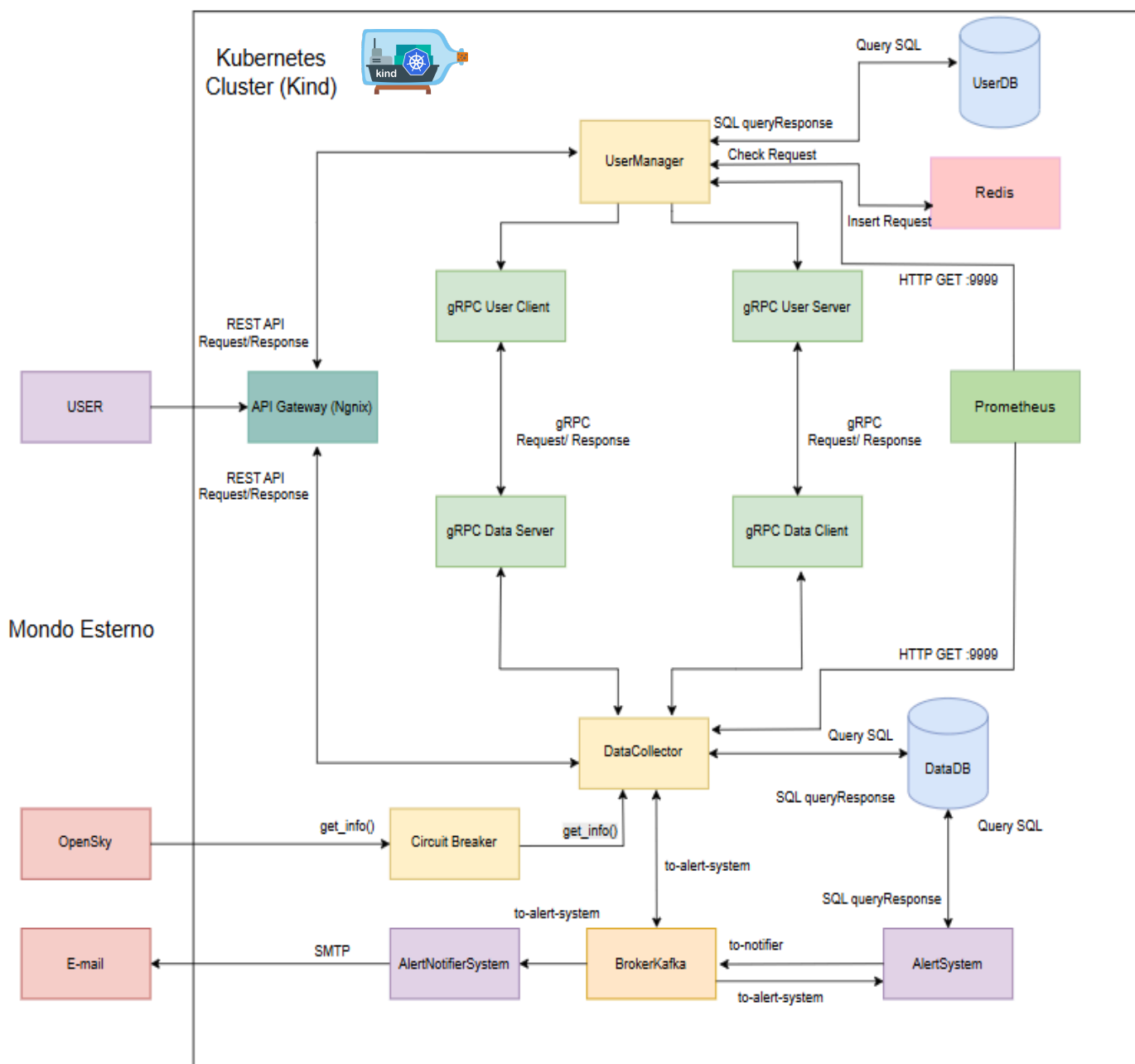
Diagramma delle Interazioni

Il diagramma mostra un'architettura a microservizi composta da **UserManager** e **DataCollector**, che espongono interfacce **REST API** verso l'utente (mondo esterno) e comunicano tra loro tramite protocollo **gRPC**.

- **Comunicazione gRPC (Bidirezionale):**
 - **User Client → Data Server:** Lo User Manager invia comandi al Data Collector (cancellazione dati).
 - **Data Client → User Server:** Il Data Collector interroga lo User Manager (verifica autenticazione).
- **Database Dedicati:** Ogni servizio gestisce il proprio archivio (**UserDB** e **DataDB**) tramite query SQL.
- **OpenSky:** Servizio esterno interrogato dal DataCollector per reperire le informazioni sui voli in tempo reale.
- **Redis:** funge da cache temporanea per intercettare i tentativi duplicati di registrazione ed evitare interrogazioni inutili al database.
- **API Gateway (Nginx):** Punto di ingresso unico che instrada le richieste REST verso i microservizi competenti (User Manager o Data Collector).
- **Broker Kafka:** Bus di messaggistica asincrono che disaccoppia la raccolta dati dalla pipeline di alerting, distribuendo i trigger di aggiornamento.
- **Alert System:** Componente Consumer/Producer che verifica le soglie nel DB e genera gli eventi di allarme.
- **Alert Notifier System:** Consumer dedicato esclusivamente all'invio delle notifiche email agli utenti.
- **Circuit Breaker:** Pattern di resilienza interposto tra il Data Collector e OpenSky; monitora i fallimenti delle chiamate esterne per prevenire sovraccarichi o attese inutili in caso di disservizi dell'API remota.
- **Servizio E-mail:** Server di posta esterno contattato via SMTP dall'Alert Notifier System per la consegna effettiva delle email agli utenti.

Componenti aggiuntive:

- **Prometheus:** Il server Prometheus agisce come un client HTTP che esegue il "pull" delle metriche periodicamente (ogni 15 secondi). Nel diagramma delle interazioni, questo si traduce in due flussi di richieste HTTP GET /metrics distinti:
 1. Verso il **Data Collector** (porta 9999): per raccogliere le metriche di business sui voli (numero download, tempi di risposta API esterna).
 2. Verso lo **User Manager** (porta 9999): per raccogliere le metriche applicative sugli utenti (latenza login/registrazione, contatori di successo/fallimento).



Lista delle API Data Collector:

Nome API	Descrizione	Messaggio di Richiesta	Messaggio di Risposta
sendInterest()	Registra l'interesse di un utente per un aeroporto, una modalità* specificata e include le soglie di allarme per l'Alert System.	SendInterestRequest email (string) token (string) airport_code (string) mode (boolean) h_value (int) l_value (int)	SendInterestResponse message (string)
delete_interest()	Rimuove l'interesse registrato di un utente.	DeleteInterestRequest email (string) token (string) airport_code (string) mode (boolean) h_value (int) l_value (int)	DeleteInterestResponse message (string)
get_info()	Recupera le informazioni sui voli per un determinato aeroporto e modalità*.	GetInfoRequest email (string) token (string) airport_code (string) mode (boolean)	GetInfoResponse count (int) voli (list of objects): partenza (string) ora_arrivo (string) ora_partenza (string) arrivo (string) codice (string)
get_last_one()	Recupera l'ultimo volo in arrivo e in partenza disponibile per l'aeroporto.	GetLastValueRequest email (string) token (string) airport_code (string)	GetLastValueResponse count (int) voli (list of objects): partenza (string) ora_arrivo (string) ora_partenza (string) arrivo (string) codice (string)
get_avgs()	Calcola la media dei voli (arrivi e partenze) negli ultimi N giorni.	GetAveragesRequest email (string) token (string) airport_code (string) n_days (int/string)	GetAveragesResponse media arrivi (double) media partenze (double)

check_time_update()	Restituisce il timestamp dell'ultimo aggiornamento dati ricevuto via Kafka.	CheckTimeRequest email (string) token (string)	CheckTimeResponse message (string)
----------------------------	---	---	--

*modalità: si intende aeroporto di arrivo o di partenza.

Lista delle API User Manager:

Nome API	Descrizione	Messaggio di Richiesta	Messaggio di Risposta
login()	Esegue l'autenticazione di un utente verificando le credenziali (email e password).	LoginRequest email (string) password (string)	LoginResponse message (string)
registrazione()	Registra un nuovo utente nel sistema salvando email, username e password (con hashing).	RegistrationRequest email (string) username (string) password (string)	RegistrationResponse message (string)
cancellazione()	Elimina definitivamente un account utente e i relativi dati (sessione e interessi), previa verifica della password.	DeleteAccountRequest email (string) password (string)	DeleteAccountResponse message (string)